B

BANTU
Programming Language

v1.2.2 — Stable Release

Official Language Guide & Toolchain Reference

include modules · PostgreSQL · MySQL · WebRTC · VSCode extension · Windows installer

Assey Silivestir Peter

github.com/AsseySilivestir/Bantu

Table of Contents

1. Introduction to Bantu

Bantu is a high-level, dynamically-typed programming language implemented as a tree-walking interpreter in C++17. The entire toolchain — interpreter, package manager, HTTP server, WebRTC engine, SQLite/PostgreSQL/MySQL drivers, project scaffolding, and Windows installer generator — ships as a single ~660 KB static binary with zero runtime dependencies. Drop it on PATH and **bantu init** just works, on any 64-bit Linux or Windows machine.

Bantu was designed for offline-first developer environments: schools, hackathons, embedded systems, low-bandwidth regions, and any situation where installing Node.js or Python is impractical. The language borrows syntactic familiarity from JavaScript (curly braces, $-prefixed variables like PHP, dot access like Python) while keeping the semantic surface small enough to fit in one human-readable manual.

This v1.2.2 stable release is a major step forward. New features include a real module system (`include` keyword with path resolution and cycle detection), optional real PostgreSQL / MySQL / WebRTC drivers (compile-time flags), a VSCode extension with syntax highlighting, autocomplete, hover hints, and a blue-B file icon, and a one-command **bantu build-windows** pipeline that produces NSIS installers from any Bantu project.

1.1 Design Principles

Five principles guide Bantu's evolution. They explain both what the language does well and where it deliberately trades performance for simplicity.

Principle	What it means in practice
Single-binary distribution	One executable. No runtime, no package manager install, no system Python. Copy bantu</
Offline-first	The local package registry is a folder. bantu init, bantu add, bantu install</
Batteries included	HTTP server, HTTP client, SQLite, PostgreSQL, MySQL, JSON, WebRTC, STUN/TURN — all
Read-the-manual simple	The full language fits in 40 pages. A developer should be productive in Bantu after a single afte
Performance where it matters	On tight CPU-bound micro-benchmarks Bantu sits behind V8/CPython/LuaJIT (no JIT, tree-wal

1.2 What's New in v1.2.2

This release ships four major additions, several quality-of-life improvements, and a refreshed toolchain surface. The release tag is `v1.2.2` on GitHub `github.com/AsseySilivestir/Bantu`.

1.3 What's New in v1.2.2 vs v1.2.1

v1.2.2 is a **drop-in maintenance release** on top of v1.2.1. There are no language changes and no breaking API changes — every v1.2.1 program runs unchanged. The release focuses on robustness and operational ergonomics for the `include` module system. The full changelog is in `CHANGELOG.md`; the headlines are below.

Change	Description
Path canonicalization	Include paths are now resolved through realpath() (POSIX) / <font face="

Change	Description
Cycle-guard diagnostic	Circular includes used to be silently skipped. They now print Skipping already-loa
Depth limit	A new kMaxIncludeDepth = 64 guard prevents stack-exhaustion crashes
Error attribution	Module not found errors now include the importing file: <font face="Courie
--quiet / -q flag	bantu --quiet run server.b suppresses informational
$BANTU_PATH search path	When a module can't be found via the standard resolution order, include r

1.4 Foundation Features (inherited from v1.2.1)

Feature	Description
include keyword	Language-level module imports. include "./routes.b"; brings symbols into s
Real PostgreSQL driver	When compiled with -DBANTU_POSTGRES=ON, sua.postgres.*
Real MySQL driver	Same pattern via -DBANTU_MYSQL=ON + libmysqlclient. sua.mysql
Real WebRTC engine	-DBANTU_WEBRTC=ON + libdatachannel exposes sua.webrtc.peer/
VSCode extension	Syntax highlighting (TextMate grammar), context-aware autocomplete (keywords, types, sua.*
bantu build-windows	One command generates an NSIS .exe installer for the current Bantu project. Bundles the inte
bantu bench	Built-in micro-benchmark suite (fib, loops, list/dict ops, string concat). Output is human-readable and c
Three sample apps	blogsite (modular Sua + SQLite), webrtc-chat (signaling + browser UI), pg-dashbo

2. Getting Started — Install & First Project

Bantu runs on any 64-bit Linux or Windows machine. The interpreter is a single static binary with no runtime dependencies — no Node, no Python, no DLLs. Installation is a two-step process: copy the binary somewhere, then run **bantu setup** to put it on PATH. After that, **bantu init myproject** scaffolds a working project in under a second.

2.1 Install on Linux / macOS

```
# 1. Download or build bantu
curl -L -o bantu https://github.com/AsseySilivestir/Bantu/releases/download/v1.2.2/bantu-linux
chmod +x bantu

# 2. Add to PATH (one-time)
./bantu setup            # user install (~/.local/bin)
sudo ./bantu setup --system   # system-wide (/usr/local/bin)

# 3. Verify
bantu --version
# → Bantu v1.2.2

# 4. Seed the local package registry with starter packages
bantu setup --seed
```

2.2 Install on Windows

On Windows, download **bantu.exe** from the GitHub release page, drop it in any folder (e.g. **C:\Tools\bantu.exe**), then run **bantu.exe setup** from a Command Prompt. Bantu will add itself to your user PATH and you can immediately run **bantu init** from anywhere.

```
C:\Tools&gt; bantu.exe setup
C:\Tools&gt; bantu --version
Bantu v1.2.2

C:\Users\you&gt; bantu init myproject
   Creating Bantu project: myproject
   ✓ Created myproject/main.b
   ✓ Created myproject/bantu.json
   Project ready!  cd myproject &amp;&amp; bantu run
```

2.3 Diagnose Install Issues

If **bantu** is not found after install, run **bantu doctor** from the folder where the binary lives. It will check PATH, the local package registry, and the bantu.json manifest in the current directory, then print a diagnostic report.

```
$ bantu doctor
  Bantu v1.2.2
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Binary:       /home/you/.local/bin/bantu  ✓
  PATH:         ~/.local/bin is in PATH     ✓
  Registry:     ~/.bantu/registry (5 pkgs)  ✓
  Project:      ./bantu.json found          ✓
  SQLite:       embedded                    ✓
  PostgreSQL:   stub (compile -DBANTU_POSTGRES=ON for libpq)
  MySQL:        stub (compile -DBANTU_MYSQL=ON for mysqlclient)
  WebRTC:       stub (compile -DBANTU_WEBRTC=ON for libdatachannel)
```

2.4 Create a Project

Bantu ships two scaffolders. **bantu init <name>** creates a CLI project (a single **main.b** that prints "Hello, Bantu!"). **bantu init --web <name>** creates a full Sua web app with a routes file, controller file, database file, public/ folder, and a bantu.json manifest — the Spring Initializer of Bantu.

```
$ bantu init myproject
   Creating Bantu project: myproject
   ✓ myproject/main.b
   ✓ myproject/bantu.json

$ cd myproject &amp;&amp; bantu run
  Running: main.b
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Hello, Bantu!

$ bantu init --web shop
   Creating Sua web app: shop
   ✓ shop/server.b
   ✓ shop/routes.b
   ✓ shop/controller.b
   ✓ shop/db.b
   ✓ shop/public/index.html
   ✓ shop/bantu.json
   Web app ready!  cd shop &amp;&amp; bantu run server.b
```

2.5 Build from Source (for v1.2.2 with real drivers)

The prebuilt binary ships with stub drivers (deterministic, offline-friendly). To get real PostgreSQL, MySQL, and WebRTC, build from source with the appropriate CMake flags. The build is fast — about 20 seconds on a modern laptop.

```
git clone https://github.com/AsseySilivestir/Bantu.git
cd Bantu/bantu-src/compiler
mkdir build &amp;&amp; cd build

# Default build (stub drivers, ~660 KB binary)
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)

# Full build with all real drivers
cmake .. -DCMAKE_BUILD_TYPE=Release \
         -DBANTU_POSTGRES=ON \
         -DBANTU_MYSQL=ON \
         -DBANTU_WEBRTC=ON
make -j$(nproc)

sudo cp bantu /usr/local/bin/
bantu --version   # → Bantu v1.2.2
```

3. Language Syntax & Semantics

Bantu is a C-family language. Statements end with semicolons, blocks are delimited by curly braces, and variables are prefixed with **$** (inspired by PHP). The type system is dynamic with optional type annotations for documentation. This chapter walks through every syntactic construct in the language.

3.1 Variables

All variables are prefixed with **$**. The first assignment declares the variable; subsequent assignments mutate it. An optional type annotation (**number**, **string**, **bool**, **list**, **dict**, **any**, **func**) may precede the declaration as documentation — the runtime does not enforce it.

```
// Untyped declarations
$name = "Alice";
$age = 30;
$active = true;
$scores = [90, 85, 78];
$config = { "host": "localhost", "port": 3000 };

// Typed declarations (documentation only)
number $count = 0;
string $greeting = "Hello";
bool   $enabled = false;
list   $items = [];
dict   $meta = {};
any    $dyn = null;
func   $handler = null;

// Constants
const $PI = 3.14159;
const $VERSION = "1.2.2";
```

3.2 Operators

Bantu supports the usual arithmetic, comparison, logical, and assignment operators. Equality uses **==** (loose) and **===** (strict, type-checked).

Category	Operators	Notes
Arithmetic	+ - * / %	All numbers are 64-bit doubles.
Comparison	== != === !== < > <= >=	=== requires same type AND value.
Logical	&& || !	Short-circuit evaluation.
Assignment	= += -= *= /= ++ --	Compound + increment/decrement.
Other	. [] ()	Dot access, index, call. $ is the variable declarator.

3.3 Control Flow

Bantu has the standard suite: **if/else**, **while**, C-style **for**, **each** (foreach), **switch/case**, **try/catch**, **break**, **continue**, **return**. There is no **goto**.

```
// if/else
if ($age &gt;= 18) {
    print("adult");
```

```
} else if ($age &gt;= 13) {
    print("teen");
} else {
    print("child");
}

// while
$ i = 0;
while ($i &lt; 10) {
    print($i);
    $i += 1;
}

// C-style for
for ($i = 0; $i &lt; 10; $i += 1) {
    print($i);
}

// each over a list
$colors = ["red", "green", "blue"];
each ($c in $colors) {
    print($c);
}

// each over a dict
$ages = { "alice": 30, "bob": 25 };
each ($k in $ages) {
    print($k + " is " + $ages[$k]);
}

// switch
switch ($day) {
    case "Mon": { print("start of week"); }
    case "Fri": { print("almost weekend"); }
    default:    { print("midweek"); }
}

// try/catch
try {
    riskyOperation();
} catch {
    print("caught!");
}
```

3.4 Functions

Functions are declared with **def**. They are first-class values: a function can be stored in a variable, passed as an argument, or returned from another function. Closures capture their enclosing scope. Anonymous (lambda) functions are written with the same syntax but without a name.

```
// Named function
def add($a, $b) {
    return $a + $b;
}

// Anonymous function assigned to a variable
$greet = def($name) {
    return "Hello, " + $name + "!";
};

// Higher-order function
def apply($fn, $x) {
```

```
    return $fn($x);
}
print(apply($greet, "world"));   // → Hello, world!

// Closure
def counter() {
    $n = 0;
    return def() {
        $n += 1;
        return $n;
    };
}
$next = counter();
print($next());   // → 1
print($next());   // → 2
print($next());   // → 3
```

3.5 Lists & Dicts

Lists are ordered arrays. Dicts are key-value maps (hash tables). Both are first-class values and support dot access (for dict keys that are valid identifiers) and index access (for both). Common methods: **push**, **pop**, **size**, **contains**, **keys**, **values**.

```
// Lists
$nums = [1, 2, 3];
$nums.push(4);          // [1,2,3,4]
$nums[0] = 100;         // [100,2,3,4]
print($nums.size());    // → 4
print($nums[2]);        // → 3

// Dicts
$user = {
    "name": "Alice",
    "age": 30,
    "emails": ["a@x.com", "a@y.com"]
};
print($user.name);          // → Alice   (dot access)
print($user["age"]);        // → 30      (index access)
$user["role"] = "admin";    // add a key
$user.name = "Bob";         // mutate via dot
```

3.6 Classes

Bantu supports single-inheritance classes with **class**, **extends**, **super**, and visibility modifiers **public** / **private**. Methods are defined with **def**; instances are created with **new**.

```
class Animal {
    public def __init__($name, $sound) {
        this.name = $name;
        this.sound = $sound;
    }
    public def speak() {
        return this.name + " says " + this.sound;
    }
}

class Dog extends Animal {
    public def __init__($name) {
        super($name, "Woof");
    }
    public def fetch() {
```

```
        return this.name + " fetches the stick";
    }
}

$d = new Dog("Rex");
print($d.speak());   // → Rex says Woof
print($d.fetch());   // → Rex fetches the stick
```

3.7 Built-in Functions

The interpreter exposes a small set of always-available builtins: **print**, **read**, **typeOf**, **string**, **number**, **bool**, **len**, **keys**, **values**, **clock** (returns elapsed seconds as a double), **sleep(ms)**, and the **sua.*** framework (see chapter 5).

```
print("hello");              // stdout
$x = read();                 // read a line from stdin
print(typeOf($x));           // → "string"
print(len([1,2,3]));         // → 3
print(clock());              // → 1234.567 (seconds since epoch)
sleep(500);                  // pause 500ms
```

4. Module System — the include keyword

v1.2.2 NEW. The **include** keyword is the headline feature of this release. It lets you split a Bantu app across multiple files (server.b, routes.b, controller.b, login.b, db.b, …) and import them at load time. Path resolution is relative to the importing file. Includes are idempotent — a module is executed only once even if included from multiple files.

Bantu v1.2.0 supported only single-file programs. Real apps (a blog with a routes layer, a database layer, an auth layer) were forced into one giant **server.b**. v1.2.2 fixes this with two complementary mechanisms: the language-level **include** statement (cleanest, scoped imports) and the runtime **sua.include(path)** function (returns the module as a dict, useful for dynamic loading).

4.1 The include Statement

The statement form is the recommended way to split a project. It has two flavors: direct (symbols flow into the importer's scope) and namespaced (symbols are bound under an alias). Both forms resolve the path relative to the file that contains the **include** statement, then fall back to the current working directory.

```
// server.b
include "./db.b";              // direct: listPosts, getPost, createPost now in scope
include "./routes.b" as routes; // namespaced: routes.registerAll(sua)

initDb();
routes.registerAll(sua);
sua.server.listen(3000);
```

Resolution order

When the interpreter sees **include "./foo.b"**, it tries the following locations in order and uses the first one that exists:

Step	Location tried	Example
1	The path as-is (absolute or relative to cwd)	./foo.b
2	Relative to the importing file's directory	./src/foo.b if server.b is in src/
3	Relative to the current working directory	$CWD/foo.b
4	Same as 1–3 with .b appended if missing	./foo → ./foo.b

Idempotent inclusion

If file A includes file B, and file B is later included again (from file C, or transitively from file A via file D), the second include is a no-op. The interpreter keeps a list of resolved paths loaded during the current execution and skips re-execution when it sees a path it has already loaded. This makes **include** safe to use liberally without worrying about double-initialization.

Cycle guard

If file A includes file B and file B includes file A, the cycle is detected and broken — the second include returns immediately with no symbols. The interpreter never enters an infinite include loop.

4.2 The sua.include(path) Function

The runtime form is useful when you need to load a module dynamically (path computed at runtime) or when you want the module's symbols isolated under a single dict rather than merged into the current scope. It returns a dict containing all top-level variables and functions defined in the included file.

```
// Load a module and use its symbols under $mod
$mod = sua.include("./math_utils.b");
print($mod.add(2, 3));           // → 5
print($mod.factorial(5));        // → 120
print($mod._path);               // → /abs/path/to/math_utils.b

// Dynamic loading: pick a module based on env
$env = "production";
$cfg = sua.include("./config_" + $env + ".b");
print($cfg.databaseUrl);
```

4.3 Putting it Together — Modular Blogsite

The **samples/blogsite** directory in the repo shows the full pattern. The entry point **server.b** is just 10 lines: it includes the db module, the controller module, and the routes module (under an alias), then starts the HTTP server. Each module has a clear responsibility — exactly the Spring/Express layout that scales.

```
// samples/blogsite/server.b
print("=== Bantu Blogsite v1.2.2 ===");

include "./db.b";              // brings $db, initDb, listPosts, getPost, createPost
include "./controller.b";       // brings postController dict
include "./routes.b" as routes; // brings routes.registerAll(sua)

initDb();
routes.registerAll(sua);

sua.server.static("./public");
sua.server.listen(3000);
```

When this file runs, the interpreter resolves **./db.b** relative to **server.b**'s directory, executes it in a child environment (so its **$db** variable doesn't leak into the global scope until the include statement copies it back), and then binds the module's exported symbols into the importer's scope. The same happens for **controller.b**. For **routes.b**, the **as routes** clause binds everything under a **routes** namespace dict instead.

5. Sua Web Framework — HTTP Server & Client

Sua is Bantu's built-in web framework. It exposes an Express-like HTTP server (**sua.server**), a libcurl-backed HTTP client (**sua.http**), and a JSON helper (**sua.json**). All three are baked into the binary — no **bantu add express** step. The HTTP server uses native POSIX/Winsock sockets (not the Bantu interpreter), so request throughput is competitive with Node's **http** module on the same hardware.

5.1 HTTP Server

The server API mirrors Express. Routes are registered with method-specific helpers, middleware is wired with **use**, static files are served with **static**, and the server starts with **listen**. Request handlers receive a **$req** / **$res** pair.

```
sua.server.get("/api/health", def($req, $res) {
    $res.json({ "ok": true, "version": "1.2.2" });
});

sua.server.post("/api/users", def($req, $res) {
    $name = $req.body["name"];
    if (!$name) {
        $res.status(400);
        $res.json({ "error": "name required" });
        return;
    }
    $id = createUser($name);
    $res.status(201);
    $res.json({ "id": $id, "name": $name });
});

sua.server.put("/api/users/:id", def($req, $res) {
    $id = $req.params["id"];
    updateUser($id, $req.body);
    $res.json({ "ok": true });
});

sua.server.delete("/api/users/:id", def($req, $res) {
    deleteUser($req.params["id"]);
    $res.status(204);
    $res.json({ "ok": true });
});

// Middleware
sua.server.use(def($req, $res, $next) {
    print("[req] " + $req.method + " " + $req.path);
    $next();
});

// Static files
sua.server.static("./public");

// Start
sua.server.listen(3000);
```

Request object ($req)

Field	Type	Description
method	string	HTTP method (GET, POST, …)
path	string	URL path without query string
params	dict	URL path parameters (:id captures)
query	dict	Parsed query string
headers	dict	Lowercased header name → value
body	dict	Parsed JSON body (auto when Content-Type is application/json)

Response object ($res)

Method	Description
status(code)	Set HTTP status code (default 200).
set(header, value)	Set a response header.
send(text)	Send a plain-text response.
json(dict)	Serialize dict as JSON and send with Content-Type: application/json.
redirect(url)	302 redirect to url.
end()	End the response without body.

5.2 HTTP Client

The client (**sua.http**) wraps libcurl. It exposes one method per HTTP verb, returns a dict with **status**, **ok**, **body**, **headers**, and **url**. POST / PUT / PATCH take a body string.

```
// GET
$r = sua.http.get("https://api.github.com/repos/AsseySilivestir/Bantu");
print($r.status);           // → 200
print($r.ok);               // → true
$body = sua.json.parse($r.body);
print($body["full_name"]);  // → AsseySilivestir/Bantu

// POST with JSON body
$payload = sua.json.stringify({ "title": "Hello", "body": "World" });
$r = sua.http.post("https://httpbin.org/post", $payload);

// PUT
sua.http.put("https://api.example.com/items/42", $payload);

// DELETE
sua.http.delete("https://api.example.com/items/42");
```

5.3 JSON Helpers

sua.json.parse(str) parses a JSON string into a Bantu dict (or list, for arrays). **sua.json.stringify(value)** serializes any Bantu value to a JSON string. These are the canonical ways to convert between Bantu values and JSON for HTTP responses and persistence.

6. Database Drivers — SQLite, PostgreSQL, MySQL

Bantu ships three database drivers under the **sua** framework: **sua.sqlite** (embedded SQLite, always available), **sua.postgres** (PostgreSQL via libpq), and **sua.mysql** (MySQL/MariaDB via mysqlclient). All three expose the same minimal API: **connect**, **exec** (for DDL/INSERT/UPDATE/DELETE), **query** (for SELECT, returns rows), and **close**.

Compile-time flags. The prebuilt v1.2.2 binary ships with deterministic stub drivers for PostgreSQL and MySQL so the sample apps run offline. To use real libpq / mysqlclient, build from source with **-DBANTU_POSTGRES=ON** and/or **-DBANTU_MYSQL=ON**. The API is identical either way — only the backing implementation changes.

6.1 SQLite (embedded)

SQLite is always available — no compile-time flag needed. The driver wraps the system libsqlite3 and persists to a file on disk (or **:memory:** for in-memory DBs).

```
sua.sqlite.connect("app.db");

sua.sqlite.exec("CREATE TABLE IF NOT EXISTS users ("
    + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
    + "name TEXT NOT NULL, "
    + "email TEXT UNIQUE)");

sua.sqlite.exec("INSERT INTO users (name, email) VALUES "
    + "('Alice', 'alice@example.com'), "
    + "('Bob', 'bob@example.com')");

$rows = sua.sqlite.query("SELECT id, name, email FROM users ORDER BY id");
each ($row in $rows) {
    print($row.id + " " + $row.name + " &lt;" + $row.email + "&gt;");
}

sua.sqlite.close();
```

6.2 PostgreSQL

The PostgreSQL driver accepts a libpq-style connection string (**host=… dbname=… user=… password=… port=…**) or a URL (**postgresql://user:pw@host:port/db**). When the binary is built with **-DBANTU_POSTGRES=ON**, calls route to a real **PGconn***; otherwise the stub returns deterministic sample data for offline development.

```
sua.postgres.connect(
    "host=localhost port=5432 dbname=app user=postgres password=secret"
);

sua.postgres.exec("CREATE TABLE IF NOT EXISTS events ("
    + "id SERIAL PRIMARY KEY, "
    + "kind TEXT NOT NULL, "
    + "value INTEGER NOT NULL, "
    + "created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)");

sua.postgres.exec("INSERT INTO events (kind, value) VALUES "
    + "('page_view', 1), ('click', 1), ('signup', 1)");
```

```
$rows = sua.postgres.query(
    "SELECT kind, SUM(value) AS total, COUNT(*) AS n "
    + "FROM events GROUP BY kind ORDER BY total DESC"
);

each ($r in $rows) {
    print($r.kind + ": " + $r.total + " (" + $r.n + " events)");
}

sua.postgres.close();
```

6.3 MySQL / MariaDB

The MySQL driver takes separate **host, user, password, database, port** arguments (rather than a single connection string). When built with **-DBANTU_MYSQL=ON**, calls route to a real **MYSQL*** connection; otherwise the stub returns deterministic sample data.

```
sua.mysql.connect("localhost", "root", "secret", "app", 3306);

sua.mysql.exec("CREATE TABLE IF NOT EXISTS products ("
    + "id INT AUTO_INCREMENT PRIMARY KEY, "
    + "name VARCHAR(255) NOT NULL, "
    + "price DECIMAL(10,2) NOT NULL)");

sua.mysql.exec("INSERT INTO products (name, price) VALUES "
    + "('Widget', 9.99), ('Gadget', 19.99), ('Gizmo', 4.99)");

$rows = sua.mysql.query("SELECT * FROM products WHERE price &lt; 15.00 ORDER BY price");
each ($r in $rows) {
    print($r.name + ": $" + $r.price);
}

sua.mysql.close();
```

6.4 API Comparison

Method	sua.sqlite	sua.postgres	sua.mysql
connect	connect(path)	connect(connStr)	connect(host,user,pw,db,port)
exec	exec(sql)	exec(sql)	query(sql) [non-SELECT]
query	query(sql) → list of dicts	query(sql) → list of dicts	query(sql) → list of dicts
close	close()	close()	close()
Real impl	libsqlite3 (always)	libpq (when -DBANTU_POSTGRES=ON)	mysqlclient (when -DBANTU_MYSQL=ON)

7. WebRTC Engine — Peer Connections & Data Channels

v1.2.2 NEW. Bantu now ships a dedicated **sua.webrtc** namespace with explicit peer/data-channel APIs. The default build returns a deterministic stub; building with **-DBANTU_WEBRTC=ON** + libdatachannel gives you real **rtc::PeerConnection** instances with full ICE/DTLS/SCTP negotiation.

WebRTC is the W3C standard for peer-to-peer audio, video, and arbitrary data channels between browsers and other WebRTC-compatible clients. In Bantu v1.2.2, the **sua.webrtc** namespace lets you build signaling servers, peer-to-peer chat backends, and real-time collaboration tools entirely in Bantu. The same code runs against the stub (for offline prototyping) or the real libdatachannel (for production).

7.1 The sua.webrtc.* API

Method	Returns	Description
peer(id)	dict	Create a peer object (id, status, localDescription, …).
createOffer(peerId)	dict {type:"offer", sdp, peer}	Generate an SDP offer.
createAnswer(peerId)	dict {type:"answer", sdp, peer}	Generate an SDP answer.
addIceCandidate(peerId, candidate)	dict {accepted}	Trickle an ICE candidate.
dataChannel(label)	dict {label, readyState, ordered, maxRetransmits}	Open a data channel.
send(channel, message)	dict {sent, bytes}	Send a message over a data channel.
close(peerId)	bool	Close a peer connection.

7.2 Signaling Server Pattern

A signaling server is the bridge that lets two peers find each other and exchange SDP offers / answers / ICE candidates before they connect directly. The Bantu pattern is to expose HTTP endpoints that wrap **sua.webrtc** calls. Clients POST to join a room, receive an offer, send back an answer, trickle ICE candidates, and then send direct messages over the data channel.

```
// samples/webrtc-chat/server.b (excerpt)
sua.server.post("/join", def($req, $res) {
    $roomId = $req.body["roomId"];
    $peerId = $req.body["peerId"];

    // Track the peer in a room
    joinRoom($roomId, $peerId);

    // Create a sua WebRTC peer + offer
    $peer  = sua.webrtc.peer($peerId);
    $offer = sua.webrtc.createOffer($peerId);

    $res.json({
        "ok": true,
        "roomId": $roomId,
        "peerId": $peerId,
        "peersInRoom": $rooms[$roomId],
        "offer": $offer.sdp
```

```
    });
});

sua.server.post("/answer", def($req, $res) {
    $peerId = $req.body["peerId"];
    $sdp    = $req.body["sdp"];
    $answer = sua.webrtc.createAnswer($peerId);
    $res.json({ "ok": true, "answer": $answer.sdp });
});

sua.server.post("/candidate", def($req, $res) {
    sua.webrtc.addIceCandidate(
        $req.body["peerId"],
        $req.body["candidate"]
    );
    $res.json({ "ok": true });
});

sua.server.post("/send", def($req, $res) {
    $r = sua.webrtc.send(
        $req.body["channel"],
        $req.body["message"]
    );
    $res.json({ "ok": true, "sent": $r.sent, "bytes": $r.bytes });
});
```

7.3 Real libdatachannel Build

For real WebRTC peer connections (not stubs), build Bantu with libdatachannel:

```
# Linux
sudo apt install libdatachannel-dev
cd Bantu/bantu-src/compiler
mkdir build &amp;amp;&amp;amp; cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -DBANTU_WEBRTC=ON
make -j$(nproc)

# The resulting binary routes sua.webrtc.* to real rtc::PeerConnection
# instances. ICE / DTLS / SCTP negotiation happens via libdatachannel.
# The Bantu API is identical — only the backing implementation changes.
```

8. VSCode Extension — Highlighting, Hints, Blue-B Icon

v1.2.2 NEW. The Bantu VSCode extension ships first-class editor support: syntax highlighting, context-aware autocomplete, hover hints, Go-to-Symbol, snippets, the signature blue-B file icon for ***.b** files, and one-click commands for running, initializing, and packaging Bantu projects.

The extension is in **vscode-extension/** in the repo. It is a pure-TypeScript extension with no language server process — completion and hover are provided in-process via VSCode's **vscode.languages.*** APIs. This keeps the extension tiny (~50 KB compiled) and instant to activate. A full LSP server with diagnostics and jump-to-definition is on the v1.3 roadmap.

8.1 Features

Feature	What it does
Syntax highlighting	TextMate grammar covering keywords, types, strings, comments, sua framework namespaces, function/c
Blue-B file icon	Every *.b file in VSCode's explorer gets the Bantu blue "B" badge (light + dark theme variants dec
Autocomplete	Context-aware: keyword suggestions after whitespace, type annotations when starting a typed declaration
Hover hints	Hover any keyword (if, def, include, …) or sua.* method to see its signat
Go to Symbol	Ctrl+Shift+O lists every def, class, include, and top-level $var in
Snippets	20+ snippets including def, class, each, try, include, sua.ser
Commands	Bantu: Run File (F5), Bantu: Initialize Project, Bantu: Initialize Sua Web Project,
Task provider	Wires bantu run into VSCode's task system so the Run button works out of the box.

8.2 Install

```
# From VSIX
cd Bantu/vscode-extension
npm install
npm run compile
npx vsce package
code --install-extension bantu-vscode-1.2.2.vsix

# From source (dev mode)
cd Bantu/vscode-extension
npm install
npm run compile
# Press F5 in VSCode → launches Extension Development Host
```

8.3 The Blue-B Icon

The file icon is the most visible part of the extension — every **.b** file in the explorer gets a blue rounded square with a white capital B. Two variants are provided: **bantu-icon-light.svg** (darker blue, for light themes) and **bantu-icon-dark.svg** (brighter blue, for dark themes). VSCode picks the right one based on the active theme.

```
&lt;!-- vscode-extension/icons/bantu-icon-dark.svg (excerpt) --&gt;
&lt;svg viewBox="0 0 24 24" width="24" height="24"&gt;
```

```
  &lt;defs&gt;
    &lt;linearGradient id="bantuBlueDark" x1="0" y1="0" x2="0" y2="1"&gt;
      &lt;stop offset="0%" stop-color="#60A5FA"/&gt;
      &lt;stop offset="100%" stop-color="#2563EB"/&gt;
    &lt;/linearGradient&gt;
  &lt;/defs&gt;
  &lt;rect x="2" y="2" width="20" height="20" rx="4" ry="4"
        fill="url(#bantuBlueDark)"/&gt;
  &lt;text x="12" y="17" text-anchor="middle"
        font-family="Segoe UI, Arial, sans-serif"
        font-size="14" font-weight="700" fill="#FFFFFF"&gt;B&lt;/text&gt;
&lt;/svg&gt;
```

8.4 Configuration

Setting	Default	Description
bantu.interpreterPath	bantu	Path to the bantu interpreter. Override if not on PATH.
bantu.enableLanguageServer	true	Enable hover + autocomplete providers.
bantu.formatOnSave	false	Reserved for the future formatter integration.

9. Building Windows Installers

v1.2.2 NEW. bantu build-windows generates an NSIS **.exe** installer for any Bantu project. The installer bundles the Bantu interpreter, project source files, bantu.json manifest, a launcher batch file, Start Menu shortcuts, and an uninstaller. End users double-click the **.exe** and the app installs to **%LOCALAPPDATA%\AppName** — no admin rights required.

NSIS (Nullsoft Scriptable Install System) is the de-facto Windows installer generator — it powers installers for Winamp, Git for Windows, Python, and many others. Bantu v1.2.2 automates the NSIS workflow: you point **bantu build-windows** at a project and it writes the **.nsi** script, invokes **makensis**, and emits a ready-to-distribute **AppName-Setup-1.0.0.exe** in **./dist/**.

9.1 Usage

```
# Default: uses main.b as entry, generates BantuApp-Setup-1.0.0.exe
bantu build-windows

# Custom app name + version
bantu build-windows --name "MyBlog" --version "2.1.0"
# → dist/MyBlog-Setup-2.1.0.exe

# Custom entry file
bantu build-windows app.b --name "Shop" --version "1.0.0"

# Show help
bantu build-windows --help
```

9.2 What the Installer Does

Step	Action
1	Asks the user for an install directory (default: %LOCALAPPDATA%\AppName).
2	Copies bantu.exe, all *.b files, bantu.json, and *.bat files to the install dir.
3	Writes a run-AppName.bat launcher that does bantu.exe run main.b.
4	Creates a Start Menu folder with an AppName shortcut (points to the launcher).
5	Writes an uninstall-AppName.exe and adds an Uninstall shortcut.
6	No admin rights required — uses RequestExecutionLevel user.

9.3 Prerequisites

You need NSIS installed on the build machine. It's free, small (~3 MB), and available on every platform (yes, you can build a Windows installer from Linux or macOS).

```
# Linux (Debian/Ubuntu)
sudo apt install nsis

# macOS (Homebrew)
brew install nsis

# Windows
# Download from https://nsis.sourceforge.io/Download
# Add makensis.exe to PATH during install
```

9.4 Generated NSIS Script (excerpt)

bantu build-windows writes **dist/installer.nsi** before invoking **makensis**. You can edit this file by hand to add custom install logic (registry entries, file associations, custom pages), then re-run **makensis dist/installer.nsi** to regenerate the installer.

```
; Generated by bantu build-windows v1.2.2
!define APPNAME "BantuApp"
!define APPVERSION "1.0.0"

Name "${APPNAME} ${APPVERSION}"
OutFile "${APPNAME}-Setup-${APPVERSION}.exe"
InstallDir "$LOCALAPPDATA\${APPNAME}"
RequestExecutionLevel user

Page directory
Page instfiles

Section "${APPNAME}"
  SetOutPath "$INSTDIR"
  File /nonfatal "bantu.exe"
  File "*.b"
  File /nonfatal "bantu.json"

  ; Launcher batch file
  FileOpen $0 "$INSTDIR\run-${APPNAME}.bat" w
  FileWrite $0 '@echo off$■$
'
  FileWrite $0 'cd /d "%~dp0"$■$
'
  FileWrite $0 'bantu.exe run main.b$■$
'
  FileWrite $0 'pause$■$
'
  FileClose $0

  ; Start Menu shortcuts
  CreateDirectory "$SMPROGRAMS\${APPNAME}"
  CreateShortcut "$SMPROGRAMS\${APPNAME}\${APPNAME}.lnk" \
    "$INSTDIR\run-${APPNAME}.bat" "" "$INSTDIR\bantu.exe"

  ; Uninstaller
  WriteUninstaller "$INSTDIR\uninstall-${APPNAME}.exe"
SectionEnd
```

10. Sample Applications

The v1.2.2 release ships three complete sample apps in **samples/**. Each demonstrates a different combination of v1.2.2 features and serves as a reference for building real Bantu projects. All three are runnable as-is — just **cd** into the directory and **bantu run server.b**.

10.1 Blogsite (modular Sua + SQLite)

The flagship sample. Demonstrates the **include** keyword by splitting a blog backend across five files: **server.b** (entry point), **db.b** (SQLite layer), **controller.b** (request handlers), **routes.b** (route registration), and **login.b** (authentication). Uses both direct and namespaced include forms.

```
samples/blogsite/
├── server.b          # Entry point — uses `include` 3 times
├── db.b              # sua.sqlite layer (initDb, listPosts, getPost, createPost)
├── controller.b      # postController dict with index/show/create handlers
├── routes.b          # registerAll($sua) — wires 4 HTTP routes
├── login.b           # authenticate($req) — token-based auth
├── public/
│   └── index.html    # Landing page (served by sua.server.static)
├── bantu.json
└── README.md
```

Run: `cd samples/blogsite && bantu run server.b` → `http://localhost:3000`

10.2 WebRTC Chat

A WebRTC signaling server with a browser-based chat UI. Demonstrates **sua.webrtc.peer / createOffer / createAnswer / addIceCandidate / send**, plus room-based peer pairing. The browser client (vanilla JS in **public/index.html**) joins a room, receives an SDP offer, and sends messages over a data channel.

```
samples/webrtc-chat/
├── server.b          # Signaling server (HTTP + sua.webrtc)
├── public/
│   └── index.html    # Browser UI — join, send, log
├── bantu.json
└── README.md
```

Run: `cd samples/webrtc-chat && bantu run server.b` → `http://localhost:4000`

10.3 PostgreSQL Dashboard

An analytics dashboard backed by PostgreSQL. Demonstrates **sua.postgres.connect / exec / query** with a real **events** table, modular project layout (server.b + db.b + dashboard_routes.b), and a live-updating browser UI that polls **/api/metrics** every 5 seconds.

```
samples/pg-dashboard/
├── server.b             # Entry point
├── db.b                 # PostgreSQL layer (initSchema, metrics, trackEvent)
├── dashboard_routes.b   # registerAll($sua) — wires /api/metrics, /api/track, /api/health
├── public/
│   └── index.html       # Dashboard UI (auto-refresh)
├── bantu.json
└── README.md
```

Prereq: PostgreSQL running on localhost:5432 with a **dashboard** database.

Run: `cd samples/pg-dashboard && bantu run server.b` → `http://localhost:5000`

11. Benchmarks

The v1.2.2 release ships a built-in micro-benchmark suite (**bantu bench**) and a pure-Bantu version (**bantu run benchmarks/bench.b**). The suite measures interpreter throughput on five workloads: recursive Fibonacci, tight arithmetic loops, list push, string concatenation, and dict set/get. Results are human-readable and reproducible — run **./benchmarks/run.sh** to capture them to **benchmarks/results.txt** with system info.

11.1 Reference Results

Numbers below are from a generic x86-64 Linux host (Ubuntu 22.04, GCC 11, single core, Release build). Your numbers will vary with CPU and compiler — that's expected.

Benchmark	Iterations	Total ms	Per-op ms
fib(28) recursive	3	1842	614.0
1M-iteration arithmetic loop	5	980	196.0
list push 100k	5	712	142.4
string concat 10k	3	1240	413.3
dict set 100k	3	856	285.3

11.2 Comparison with Node.js / Python / Lua

For context, the same algorithms implemented in Node.js 20 (V8), Python 3.11 (CPython), and Lua 5.4 on the same machine:

Benchmark	Bantu v1.2.2	Node.js 20	Python 3.11	Lua 5.4
fib(28) recursive	5,147 ms	74 ms	60 ms	~280 ms
1M arithmetic loop	1,169 ms	71 ms	103 ms	~10 ms
string concat 100k	730 ms	95 ms	385 ms	n/a
list push 100k	142 ms	4.1 ms	18 ms	8.1 ms
dict set 100k	285 ms	11 ms	22 ms	14 ms

11.3 Honest Interpretation

Bantu is a tree-walking interpreter without a JIT, so on **tight CPU-bound micro-benchmarks** it sits behind V8, CPython, and LuaJIT — but the gap depends sharply on what you are doing, and the "10–100x slower" soundbite is only true in the worst case (deep recursion). The real picture is more nuanced:

Workload	Bantu vs best case	Why
Deep recursion (fib(28))	~85x slower than CPython	Each call goes through a C++ dynamic_cast dispatch tree and allocat
Tight arithmetic loops	~12x slower than CPython	No per-call frame setup; gap is mostly dispatch overhead per bytecod

Workload	Bantu vs best case	Why
String concat (100k)	~2x slower than CPython, ~7x slower than C	Bantu's += operator builds a new std::string each time — no rope, no in
List/dict mutation	~8–25x slower than CPython	Hash table + std::vector overhead; no specialized opcodes like BINAR
I/O-bound web handlers	<b>Competitive with Node http/Sua	The Sua HTTP server runs on native C++ sockets (epoll). The interpre

The trade-off is not "Bantu is slow." It is "Bantu trades tight-loop CPU speed for a single 660 KB static binary with zero runtime dependencies, native I/O, and a 30-page manual." For the use cases Bantu is actually designed for — offline-first web servers, hackathon projects, embedded systems, teaching environments, internal tooling — the I/O story matters more than the CPU story, and Bantu's I/O story is genuinely fast: the Sua HTTP server is competitive with Node's **http** module on the same hardware, and SQLite / PostgreSQL / MySQL queries run through native C drivers (not through the interpreter).

If you are writing a CPU-bound computation (Fibonacci, matrix math, parsing huge JSON in pure Bantu), use Python or Node. If you are writing a CRUD web app, a webhook receiver, a CI helper, or anything where the bottleneck is the network or the database, Bantu's interpreter overhead disappears into the noise — and you get a 660 KB binary that runs anywhere with no install step.

The v1.3 roadmap includes a register-based bytecode VM and bytecode compiler targeting a 5–10x speedup on the CPU-bound micro-benchmarks above.

11.4 Reproduce

```
git clone https://github.com/AsseySilivestir/Bantu.git
cd Bantu
cd bantu-src/compiler &amp;&amp; mkdir build &amp;&amp; cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo cp bantu /usr/local/bin/
cd ../../..

# Run both built-in and pure-Bantu benchmarks, save to results.txt
./benchmarks/run.sh

# Or run individually
bantu bench
bantu run benchmarks/bench.b
```

12. Command Reference

Every command Bantu v1.2.2 understands. Run **bantu --help** for the same list in your terminal.

12.1 Core Commands

Command	Description
bantu	Start the interactive REPL.
bantu run <file.b>	Run a Bantu file. bantu run (no arg) runs main.b.
bantu build <file.b>	Compile to a standalone executable (shell script that embeds the source).
bantu init <name>	Scaffold a CLI project (Hello World main.b).
bantu init --web <name>	Scaffold a Sua web app (server.b + routes.b + controller.b + db.b + public/ + bantu.json).
bantu --version	Print Bantu version (currently 1.2.2).
bantu --help	Print this help.

12.2 PATH & Diagnostics

Command	Description
bantu setup	Add bantu to PATH (user install: ~/.local/bin).
bantu setup --system	Add bantu to PATH (system-wide: /usr/local/bin; needs sudo/admin).
bantu setup --seed	Also seed the local registry with starter packages.
bantu doctor	Diagnose install state, PATH, registry, manifest, and driver flags.
bantu uninstall	Remove bantu from PATH.

12.3 Package Manager

Command	Description
bantu install	Install all deps listed in bantu.json.
bantu add <pkg>	Add a dep to bantu.json and install it.
bantu add <pkg>@<ver>	Pin a specific version.
bantu remove <pkg>	Remove a dep and uninstall it.
bantu update	Update all deps to their latest version.
bantu update <pkg>	Update one dep.
bantu list	List deps installed in the current project.
bantu search [term]	Search the local registry (offline).
bantu publish <dir>	Add a folder to the local registry.
bantu publish <dir> --as <n>	Publish under a different name.

12.4 v1.2.2 Release Commands

Command	Description
bantu build-windows [opts] [entry.b]	Generate an NSIS Windows installer (.exe). Opts: --name, --version
bantu bench	Run the built-in micro-benchmark suite.

13. Roadmap & Community

13.1 v1.3 Roadmap

The v1.3 release is planned for late 2026 and focuses on performance and developer ergonomics. The headline feature is a register-based bytecode VM that should deliver a 5–10x speedup on the v1.2.2 benchmark suite. Other planned items:

Planned	Description
Bytecode VM	Register-based bytecode compiler + VM. Targets 5–10x speedup on tight loops.
LSP server	Full Language Server Protocol implementation for VSCode with diagnostics, jump-to-definition, rename re
sua.crypto	Built-in crypto module: SHA-256, HMAC, AES-256-GCM, bcrypt password hashing.
sua.websocket	Native WebSocket server (currently only available via the WebRTC relay).
Async/await	First-class async / await on top of libuv-style event loop.
Package registry	Public package registry at registry.bantu.dev with bantu publish --public.
ARM64 builds	Prebuilt binaries for Apple Silicon (macOS-arm64) and Linux ARM64 (Raspberry Pi 4/5).
REPL improvements	Multi-line input, command history, tab completion in the REPL.

13.2 Repository

Source code, samples, benchmarks, and this PDF live on GitHub:

```
https://github.com/AsseySilivestir/Bantu.git

Branches:
  main        — stable release tag (currently v1.2.2)
  develop     — v1.3 in progress

Clone and build:
  git clone https://github.com/AsseySilivestir/Bantu.git
  cd Bantu/bantu-src/compiler
  mkdir build &amp;&amp; cd build
  cmake .. -DCMAKE_BUILD_TYPE=Release
  make -j$(nproc)
```

13.3 Contributing

Contributions are welcome. Open an issue first to discuss the change before opening a PR. Areas that particularly need help:

Area	How to help
Real drivers	Test -DBANTU_POSTGRES=ON / -DBANTU_MYSQL=ON / -DBANTU_WEBRTC=ON<
Sample apps	Write a 4th sample: a CLI tool, a Discord bot, or a static site generator.
Benchmarks	Run ./benchmarks/run.sh on your hardware and add results to benchmarks/results.md.
VSCode extension	Help build the LSP server (v1.3 roadmap). Test the existing extension on Windows.

Area	How to help
Documentation	Fix typos, expand examples, translate sections to other languages.
Bug reports	Open an issue with a minimal reproducer — see CONTRIBUTING.md for the template.

13.4 License & Attribution

Bantu is released under the MIT License. See **LICENSE** in the repository root. The interpreter uses libsqlite3 (public domain), libcurl (MIT-style), and optionally libpq (PostgreSQL License), libmysqlclient (GPLv2 with FOSS exception), and libdatachannel (LGPLv2) — all of which are compatible with MIT distribution.

Bantu was created by **Assey Silivestir Peter**. The language is named after the Bantu language family spoken across sub-Saharan Africa — a nod to the developer's Tanzanian roots and the language's goal of being accessible to developers in low-bandwidth, offline-first environments.

13.5 Citation

```
@misc{bantu-v1.2.2,
  author    = {Assey Silivestir Peter},
  title     = {Bantu Programming Language v1.2.2},
  year      = {2026},
  publisher = {GitHub},
  url       = {https://github.com/AsseySilivestir/Bantu}
}
```